

Proyecto 1



Desarrollo de una FFT de 1024 puntos mediante mariposas

Procesamiento digital de señales

División de Ingeniería Eléctrica

Cruces Nuñez Claudia Meera - 320277038

Tellez Olmedo Eduardo Ivan - 321134873

7 de octubre del 2026

2027-1

Índice

1	Introducción	1
2	Planteamiento del problema	2
2.1	La DFT de 1024 puntos	2
2.2	Factores de giro para $N = 1024$	2
2.3	Descomposición en 10 etapas	3
3	Desarrollo por mariposas	3
3.1	Operación básica de mariposa	3
3.2	Agrupamiento de las muestras en cada etapa	5
3.3	Uso de los factores de giro	6
3.4	Flujo de datos de la entrada a la salida	7
4	Estructura de las 10 etapas	8
4.1	Distancia entre los elementos de una mariposa	8
4.2	Organización en bloques	9
4.3	Resumen de las 10 etapas	9
5	Ordenamiento de datos	11
5.1	La regla de inversión de bits	11
5.2	Aplicación a $N = 1024$	12
5.3	Decimación en frecuencia	12
6	Complejidad computacional	13
6.1	Costo de la DFT directa	13
6.2	Costo de la FFT	13
6.3	Comparación para $N = 1024$	13
6.4	El ahorro crece con N	13
7	Implementación en Código y Validación Experimental	14
7.1	Estructura de la Función <code>mi_fft_radix2</code>	14
7.2	Caso de Prueba: Señal Multitono y Comparación de Resultados	16
7.3	Análisis de Resultados	17
8	Conclusiones	18

1. Introducción

La Transformada Discreta de Fourier (DFT) permite obtener el espectro discreto de una secuencia $x(n)$ de longitud finita N . Siguiendo la notación de [3], se define como

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn}, \quad k = 0, 1, \dots, N-1, \quad (1)$$

donde $W_N^k = e^{-j2\pi k/N}$ son los factores de giro (*twiddle factors*). La DFT es reversible y periódica, con período igual a la longitud N de la secuencia.

Diferencia entre DFT y FFT

La DFT es una *transformación*: a una secuencia finita de N muestras en el tiempo le asocia N muestras de su espectro, equiespaciadas en frecuencia. La Transformada Rápida de Fourier (FFT), en cambio, no es una transformada distinta, sino un *algoritmo* para calcular la misma DFT de forma eficiente [3]. Su resultado es idéntico al de evaluar la ecuación (1) directamente. La diferencia está únicamente en la organización de los cálculos, que reduce de manera drástica el número de sumas y multiplicaciones requeridas. El algoritmo que popularizó este enfoque fue el de Cooley y Tukey [2].

¿Por qué la FFT reduce la complejidad?

Para calcular cada valor $X(k)$ con la ecuación (1) se necesitan N multiplicaciones complejas. Como hay N valores de k , la DFT directa requiere

$$N \cdot N = N^2 \text{ multiplicaciones complejas.} \quad (2)$$

La FFT aprovecha las propiedades de simetría y periodicidad de los factores de giro para dividir el cálculo sucesivamente a la mitad [3, 4]. Cuando N es potencia de dos, esta división puede repetirse $\log_2 N$ veces, lo que da lugar a $\log_2 N$ etapas. En cada etapa se realizan $N/2$ multiplicaciones complejas, por lo que el costo total es

$$\frac{N}{2} \log_2 N \text{ multiplicaciones complejas.} \quad (3)$$

Para $N = 1024 = 2^{10}$ se tiene $\log_2 1024 = 10$ etapas, y por lo tanto:

$$\text{DFT directa: } N^2 = 1024^2 = 1\,048\,576, \quad (4)$$

$$\text{FFT: } \frac{N}{2} \log_2 N = 512 \times 10 = 5\,120. \quad (5)$$

Es decir, la FFT requiere aproximadamente 200 veces menos multiplicaciones que la DFT directa. Esta comparación se analiza con detalle en la Sección 6.

2. Planteamiento del problema

2.1 La DFT de 1024 puntos

Se desea calcular la DFT de una secuencia $x(n)$ de $N = 1024$ muestras. Sustituyendo $N = 1024$ en la ecuación (1) se obtiene

$$X(k) = \sum_{n=0}^{1023} x(n) W_{1024}^{kn}, \quad k = 0, 1, \dots, 1023, \quad (6)$$

donde:

- $x(n)$ es la muestra de la señal en el instante discreto n , con $n = 0, 1, \dots, 1023$;
- $X(k)$ es el coeficiente espectral asociado a la frecuencia discreta $\omega_k = 2\pi k/1024$;
- W_{1024}^{kn} es el factor de giro que relaciona la muestra n con la frecuencia k .

La DFT recibe entonces 1024 muestras en el tiempo y entrega 1024 coeficientes en frecuencia. Cada coeficiente $X(k)$ es una suma de 1024 productos, por lo que el cálculo directo de los 1024 coeficientes requiere $1024^2 = 1\,048\,576$ multiplicaciones complejas.

2.2 Factores de giro para $N = 1024$

Los factores de giro para este problema son

$$W_{1024}^k = e^{-j \frac{2\pi k}{1024}} = \cos\left(\frac{2\pi k}{1024}\right) - j \sin\left(\frac{2\pi k}{1024}\right). \quad (7)$$

Geométricamente, cada W_{1024}^k es un punto sobre la circunferencia unitaria del plano complejo, y entre un factor y el siguiente hay un giro de $2\pi/1024$ radianes. De aquí su nombre: multiplicar por W_{1024}^k equivale a *girar* un número complejo sin cambiar su magnitud.

Los factores de giro cumplen dos propiedades fundamentales, que son la base de la eficiencia de la FFT [3]:

$$W_N^{k+N} = W_N^k \quad (\text{periodicidad}), \quad (8)$$

$$W_N^{k+N/2} = -W_N^k \quad (\text{simetría}). \quad (9)$$

Para $N = 1024$ estas propiedades toman la forma

$$W_{1024}^{k+1024} = W_{1024}^k, \quad (10)$$

$$W_{1024}^{k+512} = -W_{1024}^k. \quad (11)$$

La periodicidad indica que, aunque en la ecuación (6) aparecen productos kn muy grandes, sólo existen 1024 factores de giro distintos. La simetría indica que la mitad de ellos son el negativo de la otra mitad: girar 512 posiciones equivale a dar media vuelta (π radianes) sobre la circunferencia unitaria, lo que invierte el signo. Por lo tanto, la FFT sólo necesita conocer los 512 factores $W_{1024}^0, W_{1024}^1, \dots, W_{1024}^{511}$ [3].

2.3 Descomposición en 10 etapas

El algoritmo FFT radix-2 requiere que la longitud de la secuencia sea una potencia de dos, ya que en cada etapa el problema se divide a la mitad. En este caso la condición se cumple, pues

$$N = 1024 = 2^{10}. \quad (12)$$

La secuencia puede dividirse a la mitad diez veces ($1024 \rightarrow 512 \rightarrow 256 \rightarrow \dots \rightarrow 2 \rightarrow 1$), por lo que la FFT de 1024 puntos se implementa en

$$\log_2 1024 = 10 \text{ etapas}. \quad (13)$$

Siguiendo el procedimiento de decimación en el tiempo, la primera etapa calcula DFT de 2 puntos. Cada etapa siguiente combina pares de DFT para formar DFT del doble de tamaño, hasta obtener en la última etapa la DFT completa de 1024 puntos [3].

Tabla 1: DFT calculadas en cada etapa de la FFT de 1024 puntos.

Etapas	Número de DFT	Tamaño de cada DFT	Mariposas en la etapa
1	512	2	512
2	256	4	512
3	128	8	512
4	64	16	512
5	32	32	512
6	16	64	512
7	8	128	512
8	4	256	512
9	2	512	512
10	1	1024	512

Se observa que, aunque el número y el tamaño de las DFT cambian de una etapa a otra, el número de mariposas por etapa se mantiene constante e igual a $N/2 = 512$. La construcción de cada mariposa y la organización detallada de las etapas se desarrollan en las Secciones 3 y 4.

3. Desarrollo por mariposas

3.1 Operación básica de mariposa

La mariposa es el bloque elemental con el que se construye toda la FFT. Su funcionamiento es sencillo: recibe dos números complejos, a los que llamaremos a y b , y entrega otros dos. Su nombre proviene de la forma de su diagrama, mostrado en la Figura 1, donde las líneas que se cruzan en el centro recuerdan las alas de una mariposa.

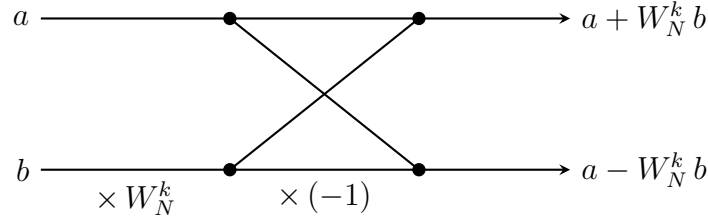


Figura 1: Diagrama de la operación mariposa.

El diagrama se lee de izquierda a derecha:

- El valor b , que entra por abajo, se multiplica primero por el factor de giro W_N^k .
- Después, cada valor se reparte por dos caminos: uno recto y uno diagonal. Por eso las líneas se cruzan en el centro.
- En cada punto de la derecha se suman los valores que llegan a él.
- En el camino inferior hay un factor -1 , que cambia el signo antes de sumar.

Siguiendo los caminos, a la salida superior llega a más $W_N^k b$, y a la salida inferior llega a menos $W_N^k b$. Es decir, la mariposa calcula

$$\text{salida superior} = a + W_N^k b, \quad \text{salida inferior} = a - W_N^k b. \quad (14)$$

La ventaja de esta operación está en que el producto $W_N^k b$ se calcula una sola vez y se aprovecha dos veces: una sumándolo y otra restándolo. Así, cada mariposa entrega dos resultados con sólo **una multiplicación compleja y dos sumas complejas**. Esta reutilización es posible gracias a la propiedad de simetría de los factores de giro, ecuación (9) [3].

Ejemplo. El caso más simple es la DFT de 2 puntos. Para la secuencia $x(n) = \{3, 1\}$ se toma $a = x(0) = 3$, $b = x(1) = 1$ y el factor de giro $W_2^0 = 1$. La mariposa entrega

$$X(0) = 3 + 1 = 4, \quad X(1) = 3 - 1 = 2, \quad (15)$$

que es exactamente la DFT de 2 puntos de la secuencia. Es decir, una DFT de 2 puntos es una sola mariposa.

Papel de la mariposa dentro de la FFT. En la FFT, las entradas a y b no son muestras cualquiera. Son los resultados de dos DFT más pequeñas: una calculada con las muestras de índice par y otra con las de índice impar. Al combinarlas mediante mariposas se obtiene una DFT del doble de tamaño [3]. Repitiendo este proceso etapa por etapa, de 2 a 4 puntos, de 4 a 8, y así sucesivamente, se llega a la DFT completa de 1024 puntos, como se resumió en la Tabla 1.

3.2 Agrupamiento de las muestras en cada etapa

Para que cada mariposa reciba los dos valores que debe combinar, las muestras de la entrada deben agruparse de una forma específica. Este agrupamiento se obtiene dividiendo la secuencia repetidamente en muestras de índice par y de índice impar, que es el principio de la decimación en el tiempo [3].

P Para explicar el procedimiento se usa una secuencia de $N = 8$ muestras [1], ya que el caso de 1024 sigue exactamente la misma lógica.

División de la secuencia. Primero, las 8 muestras se separan en dos grupos de 4: las de índice par y las de índice impar. Después, cada uno de esos grupos se vuelve a separar en pares e impares, lo que da cuatro grupos de 2 muestras. La Tabla 2 muestra este proceso.

Tabla 2: División sucesiva en pares e impares para $N = 8$.

Secuencia original	$x(0)$	$x(1)$	$x(2)$	$x(3)$	$x(4)$	$x(5)$	$x(6)$	$x(7)$
Primera división	$x(0)$	$x(2)$	$x(4)$	$x(6)$	$x(1)$	$x(3)$	$x(5)$	$x(7)$
Segunda división	$x(0)$	$x(4)$	$x(2)$	$x(6)$	$x(1)$	$x(5)$	$x(3)$	$x(7)$

Al terminar la división, las muestras quedan en un orden distinto al original:

$$x(0), x(4), x(2), x(6), x(1), x(5), x(3), x(7).$$

Este es el orden en que deben entrar a la FFT. La regla que genera este orden, conocida como inversión de bits, se explica en la Sección 5.

Reconstrucción por etapas. Una vez ordenadas las muestras, la FFT recorre el camino inverso: va juntando los grupos pequeños hasta formar la DFT completa.

- **Etapas 1:** se toman las muestras de dos en dos, por ejemplo $x(0)$ con $x(4)$, y con una mariposa se calcula la DFT de cada par. Se obtienen 4 DFT de 2 puntos.
- **Etapas 2:** se juntan las DFT de 2 puntos de dos en dos para formar DFT de 4 puntos. Se obtienen 2 DFT de 4 puntos: una con las muestras pares y otra con las impares.
- **Etapas 3:** se juntan las dos DFT de 4 puntos para formar la DFT completa de 8 puntos.

En cada etapa, el tamaño de los grupos se duplica y su número se reduce a la mitad. Sin embargo, todas las muestras participan en todas las etapas, por lo que siempre se realizan $N/2 = 4$ mariposas por etapa.

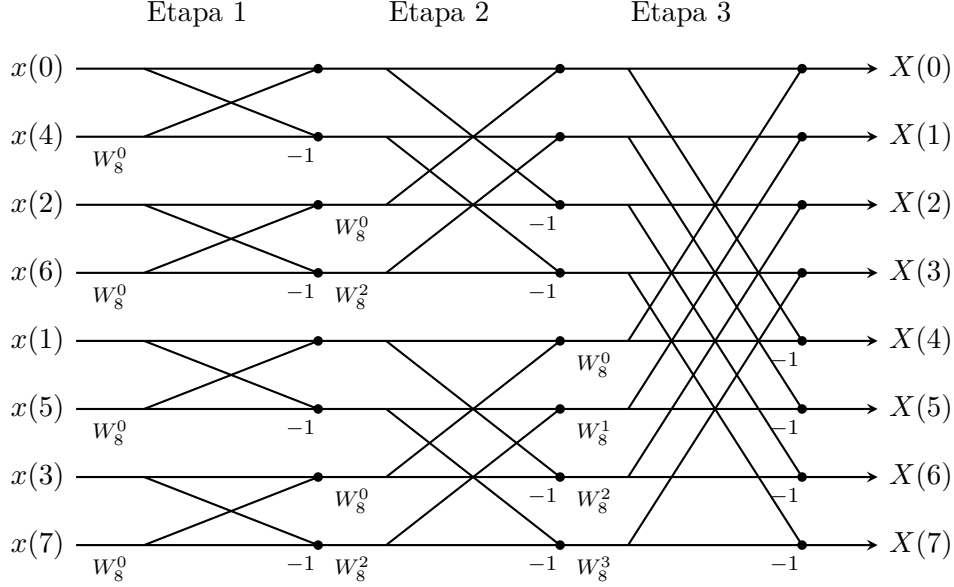


Figura 2: Diagrama de flujo de la FFT radix-2 con decimación en el tiempo para $N = 8$. Cada cruce corresponde a una mariposa (Figura 1).

Para 1024 muestras el procedimiento es idéntico. La secuencia se divide en pares e impares diez veces, y después la FFT reconstruye la DFT en diez etapas. Los grupos crecen de 2 a 4, a 8, y así sucesivamente hasta 1024, como se resumió en la Tabla 1. La forma precisa en que se eligen las parejas de cada mariposa dentro de cada etapa se detalla en la Sección 4.

3.3 Uso de los factores de giro

En cada mariposa, el valor que entra por abajo se multiplica por un factor de giro W_N^k antes de combinarse con el de arriba (Figura 1). Como se vio en la Sección 2, multiplicar por W_N^k equivale a girar un número complejo sobre el plano sin cambiar su magnitud. Este giro es lo que permite que, al sumar y restar, cada componente de frecuencia quede en su lugar correcto.

Un punto importante es que no todas las mariposas usan el mismo factor de giro. El factor depende de la etapa y de la posición de la mariposa dentro de ella. Esto se aprecia en el diagrama de $N = 8$ (Figura 2):

- **Etapa 1:** todas las mariposas usan $W_8^0 = 1$.
- **Etapa 2:** se usan dos factores, W_8^0 y W_8^2 .
- **Etapa 3:** se usan cuatro factores, W_8^0 , W_8^1 , W_8^2 y W_8^3 .

Se observa un patrón: en cada etapa el número de factores distintos se duplica, y estos quedan cada vez más juntos sobre la circunferencia unitaria. De forma general, en la etapa s se usan 2^{s-1} factores distintos, dados por

$$W_N^{r \cdot N/2^s}, \quad r = 0, 1, \dots, 2^{s-1} - 1. \quad (16)$$

Por ejemplo, para $N = 8$ y $s = 2$ el exponente avanza de $8/4 = 2$ en 2, lo que da W_8^0 y W_8^2 , en concordancia con el diagrama.

Factores de giro para $N = 1024$. Aplicando la ecuación (16) a la FFT de 1024 puntos se obtiene la Tabla 3.

Tabla 3: Factores de giro utilizados en cada etapa de la FFT de 1024 puntos.

Etapas	Factores distintos	Factores utilizados
1	1	W_{1024}^0
2	2	$W_{1024}^0, W_{1024}^{256}$
3	4	$W_{1024}^0, W_{1024}^{128}, W_{1024}^{256}, W_{1024}^{384}$
4	8	$W_{1024}^0, W_{1024}^{64}, \dots, W_{1024}^{448}$
5	16	$W_{1024}^0, W_{1024}^{32}, \dots, W_{1024}^{480}$
6	32	$W_{1024}^0, W_{1024}^{16}, \dots, W_{1024}^{496}$
7	64	$W_{1024}^0, W_{1024}^8, \dots, W_{1024}^{504}$
8	128	$W_{1024}^0, W_{1024}^4, \dots, W_{1024}^{508}$
9	256	$W_{1024}^0, W_{1024}^2, \dots, W_{1024}^{510}$
10	512	$W_{1024}^0, W_{1024}^1, \dots, W_{1024}^{511}$

De la tabla se desprenden dos observaciones:

- Todos los factores de todas las etapas pertenecen al mismo conjunto de 512 valores, W_{1024}^0 a W_{1024}^{511} , identificado en la Sección 2. Por ello, en una implementación práctica estos valores se calculan una sola vez, se guardan en una tabla y cada etapa toma de ahí los que necesita.
- Algunos factores son triviales: $W_{1024}^0 = 1$ y $W_{1024}^{256} = e^{-j\pi/2} = -j$. Multiplicar por ellos no requiere una multiplicación real, ya que sólo se conserva el valor o se intercambian la parte real y la imaginaria con un cambio de signo. Por ejemplo, en la etapa 1 todas las mariposas usan $W_{1024}^0 = 1$, así que en la práctica se reducen a una suma y una resta.

3.4 Flujo de datos de la entrada a la salida

Con los elementos anteriores es posible describir el recorrido completo de los datos a través de la FFT. El diagrama de la Figura 2 se lee de izquierda a derecha, y en él se distinguen tres partes:

1. **Entrada reordenada.** Las muestras no entran en su orden natural, sino en el orden obtenido al dividir la secuencia repetidamente en pares e impares (Sección 3.2). Para $N = 8$ este orden es $x(0), x(4), x(2), x(6), x(1), x(5), x(3), x(7)$.
2. **Etapas de mariposas.** Cada etapa recibe los N valores que entrega la etapa anterior, aplica $N/2$ mariposas con sus factores de giro correspondientes (Sección 3.3) y entrega N valores nuevos a la etapa siguiente.

3. **Salida en orden natural.** Al terminar la última etapa, los coeficientes de la DFT aparecen directamente en orden: $X(0), X(1), \dots, X(N-1)$.

Cada salida depende de todas las entradas. En la DFT, cada coeficiente $X(k)$ depende de las N muestras de entrada. En la FFT esto se cumple de forma gradual. Después de la etapa 1, cada valor contiene información de 2 muestras; después de la etapa 2, de 4 muestras; después de la etapa 3, de 8. Es decir, el alcance de cada valor se duplica en cada etapa. Para $N = 8$, al terminar la tercera etapa cada salida ya contiene información de las 8 muestras. Para $N = 1024$, esto ocurre justo al terminar la décima etapa, ya que $2^{10} = 1024$.

Ejemplo: el coeficiente $X(0)$. Si se sigue en el diagrama el camino que llega a $X(0)$, se observa que en todas las etapas se toma la salida superior de la mariposa, donde sólo hay sumas, y que todos los factores de giro en ese camino son $W_8^0 = 1$. Por lo tanto,

$$X(0) = x(0) + x(1) + x(2) + \dots + x(7), \quad (17)$$

que coincide con lo que da la definición de la DFT para $k = 0$, donde todos los factores de giro valen $W_N^0 = 1$.

En el diagrama se observa que las dos salidas de cada mariposa ocupan las mismas posiciones, o renglones, que sus dos entradas. Esto significa que, una vez calculada una mariposa, sus entradas ya no se necesitan y pueden sobrescribirse con los resultados. Gracias a ello, toda la FFT puede calcularse usando un solo arreglo de N posiciones de memoria, sin necesidad de arreglos adicionales para cada etapa. Esta característica es especialmente valiosa en implementaciones en hardware o en procesadores digitales de señales con memoria limitada.

El flujo completo de la FFT de 1024 puntos puede resumirse como:

1. Reordenar las 1024 muestras de entrada por inversión de bits (Sección 5).
2. Para cada etapa $s = 1, 2, \dots, 10$: aplicar 512 mariposas, usando los factores de giro de la Tabla 3.
3. Leer los 1024 coeficientes $X(k)$ en orden natural.

4. Estructura de las 10 etapas

En la Sección 3 se describió qué hace cada mariposa y cómo fluyen los datos. En esta sección se detalla cómo se organizan las mariposas dentro de cada una de las 10 etapas, es decir, qué posiciones del arreglo se combinan entre sí.

4.1 Distancia entre los elementos de una mariposa

Cada mariposa combina dos valores del arreglo. La separación entre las posiciones de esos dos valores se llama *distancia*, y es la característica que distingue a una etapa de

otra. Esto se observa en el diagrama de $N = 8$ (Figura 2):

- **Etapla 1:** se combinan posiciones vecinas, como 0 con 1 o 2 con 3. La distancia es 1.
- **Etapla 2:** se combinan posiciones separadas por 2, como 0 con 2 o 1 con 3. La distancia es 2.
- **Etapla 3:** se combinan posiciones separadas por 4, como 0 con 4 o 1 con 5. La distancia es 4.

Es decir, la distancia se duplica en cada etapa. En general, en la etapa s la distancia es

$$d_s = 2^{s-1}. \quad (18)$$

Es importante notar que se habla de *posiciones* dentro del arreglo y no de los índices originales de las muestras, ya que la entrada fue reordenada previamente.

4.2 Organización en bloques

En cada etapa, el arreglo se divide en bloques de tamaño $2d_s$. Dentro de cada bloque, cada posición de la primera mitad se combina con la posición que está d_s lugares más adelante, en la segunda mitad. Así, cada bloque contiene d_s mariposas.

Por ejemplo, en la etapa 3 de la FFT de 1024 puntos la distancia es $d_3 = 4$, por lo que los bloques son de 8 posiciones:

- Bloque 1 (posiciones 0 a 7): mariposas (0, 4), (1, 5), (2, 6) y (3, 7).
- Bloque 2 (posiciones 8 a 15): mariposas (8, 12), (9, 13), (10, 14) y (11, 15).

Y así sucesivamente, hasta completar 128 bloques. En total se tienen $128 \times 4 = 512$ mariposas, como se esperaba.

Los factores de giro siguen el mismo orden dentro de cada bloque: la primera mariposa del bloque usa W_N^0 , la segunda el siguiente factor de la etapa según la ecuación (16), y así sucesivamente. Todos los bloques de una misma etapa repiten la misma secuencia de factores. En el diagrama de $N = 8$, por ejemplo, la etapa 2 tiene dos bloques y ambos usan W_8^0 y W_8^2 , en ese orden.

4.3 Resumen de las 10 etapas

La Tabla 4 resume la organización de cada etapa de la FFT de 1024 puntos.

Tabla 4: Organización de las mariposas en cada etapa de la FFT de 1024 puntos.

Etapa	Distancia d_s	Tamaño de bloque	Número de bloques	Mariposas por bloque
1	1	2	512	1
2	2	4	256	2
3	4	8	128	4
4	8	16	64	8
5	16	32	32	16
6	32	64	16	32
7	64	128	8	64
8	128	256	4	128
9	256	512	2	256
10	512	1024	1	512

De la tabla se observa el patrón general de la FFT:

- En las primeras etapas hay muchos bloques pequeños, y en las últimas pocos bloques grandes.
- Aunque la organización cambia, el número total de mariposas por etapa siempre es 512, ya que $(\text{número de bloques}) \times (\text{mariposas por bloque}) = 512$ en todas las filas.
- En la última etapa hay un solo bloque y cada posición p se combina con la posición $p + 512$. Esto corresponde exactamente a la mariposa de la Sección 3.1, que relaciona $X(k)$ con $X(k + N/2)$.

La Figura 3 resume gráficamente esta estructura para la FFT de 1024 puntos.

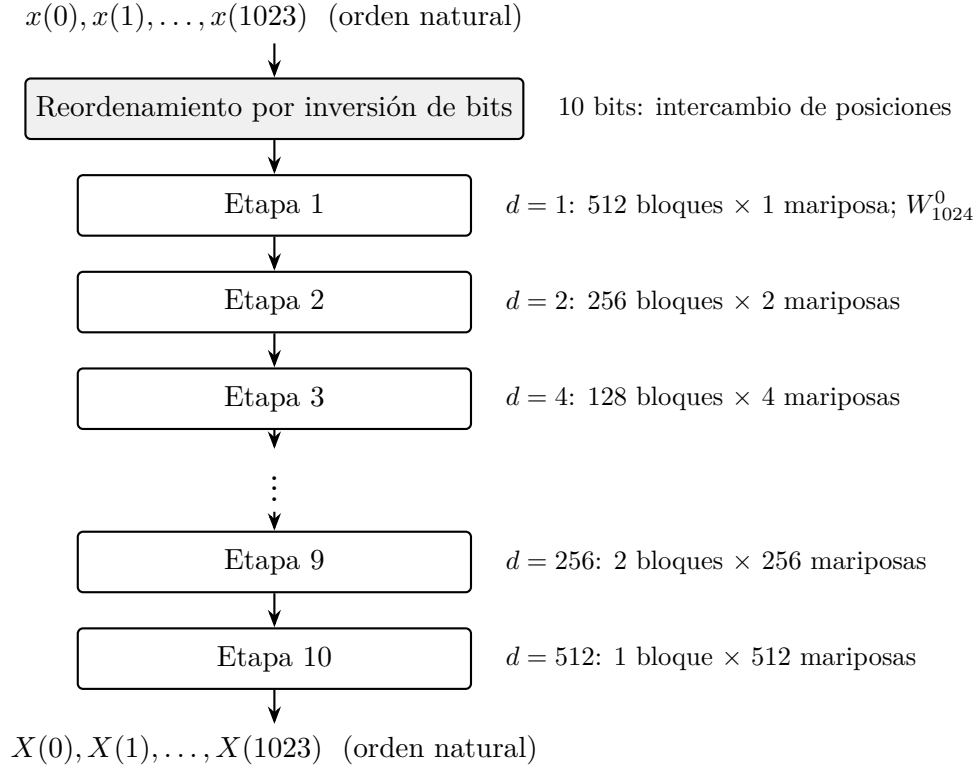


Figura 3: Diagrama general de bloques de la FFT radix-2 con decimación en el tiempo para $N = 1024$.

5. Ordenamiento de datos

5.1 La regla de inversión de bits

El procedimiento consiste en tres pasos:

1. Escribir el índice de cada posición en binario, usando $\log_2 N$ bits.
2. Invertir el orden de los bits, leyéndolos de derecha a izquierda.
3. Convertir el resultado de nuevo a decimal. Ese número indica qué muestra va en esa posición.

Para $N = 8$ se usan $\log_2 8 = 3$ bits, y el resultado se muestra en la Tabla 5.

Tabla 5: Inversión de bits para $N = 8$.

Posición	Binario	Bits invertidos	Muestra que entra
0	000	000	$x(0)$
1	001	100	$x(4)$
2	010	010	$x(2)$
3	011	110	$x(6)$
4	100	001	$x(1)$
5	101	101	$x(5)$
6	110	011	$x(3)$
7	111	111	$x(7)$

La última columna reproduce exactamente el orden obtenido en la Tabla 2 mediante divisiones sucesivas.

5.2 Aplicación a $N = 1024$

Para la FFT de 1024 puntos se usan $\log_2 1024 = 10$ bits. Algunos ejemplos:

Tabla 6: Ejemplos de inversión de bits para $N = 1024$.

Posición	Binario (10 bits)	Bits invertidos	Muestra que entra
0	0000000000	0000000000	$x(0)$
1	0000000001	1000000000	$x(512)$
2	0000000010	0100000000	$x(256)$
3	0000000011	1100000000	$x(768)$
4	0000000100	0010000000	$x(128)$
512	1000000000	0000000001	$x(1)$

Una propiedad útil es que la inversión de bits sólo *intercambia* muestras por parejas. Si la posición p recibe la muestra q , entonces la posición q recibe la muestra p . Por ejemplo, $x(1)$ y $x(512)$ simplemente se intercambian. Además, los índices cuyo binario se lee igual en ambos sentidos, como 0 o 1023, no se mueven. Por ello, el reordenamiento puede hacerse en el mismo arreglo, intercambiando valores, sin necesidad de memoria adicional,

5.3 Decimación en frecuencia

En este reporte se utilizó la decimación en el tiempo, donde el reordenamiento se aplica a la *entrada* y la salida aparece en orden natural. En la variante de decimación en frecuencia ocurre lo contrario: la entrada se procesa en orden natural y los coeficientes $X(k)$ aparecen en orden de inversión de bits a la *salida*, por lo que el reordenamiento se aplica al final [3]. En ambos casos el número de operaciones es el mismo.

6. Complejidad computacional

La complejidad computacional de un algoritmo indica cuántas operaciones requiere para resolver un problema y, sobre todo, cómo crece ese número cuando el tamaño del problema aumenta. En esta sección se compara el número de operaciones que requiere la DFT calculada directamente con el que requiere la FFT radix-2.

6.1 Costo de la DFT directa

Como se vio en la Introducción, cada coeficiente $X(k)$ es una suma de N productos. Calcular un coeficiente requiere entonces N multiplicaciones complejas y $N - 1$ sumas complejas. Como hay N coeficientes, el costo total es

$$\text{DFT directa:} \quad N^2 \text{ multiplicaciones,} \quad N(N - 1) \text{ sumas.} \quad (19)$$

6.2 Costo de la FFT

Como se resume en la Figura 3, la FFT tiene $\log_2 N$ etapas con $N/2$ mariposas cada una, y cada mariposa requiere una multiplicación compleja y dos sumas complejas (Sección 3.1). Por lo tanto, el costo total es

$$\text{FFT:} \quad \frac{N}{2} \log_2 N \text{ multiplicaciones,} \quad N \log_2 N \text{ sumas.} \quad (20)$$

6.3 Comparación para $N = 1024$

Sustituyendo $N = 1024$ y $\log_2 N = 10$ en las expresiones anteriores se obtiene la Tabla 7.

Tabla 7: Número de operaciones complejas para $N = 1024$.

Operación	DFT directa	FFT	Reducción
Multiplicaciones	1 048 576	5 120	≈ 205 veces
Sumas	1 047 552	10 240	≈ 102 veces

Es decir, la FFT realiza sólo alrededor del 0.5 % de las multiplicaciones que requiere la DFT directa, y obtiene exactamente el mismo resultado.

6.4 El ahorro crece con N

La razón entre el número de multiplicaciones de ambos métodos es

$$\frac{N^2}{\frac{N}{2} \log_2 N} = \frac{2N}{\log_2 N}. \quad (21)$$

Como N crece mucho más rápido que $\log_2 N$, la ventaja de la FFT aumenta conforme la secuencia es más larga. Por ejemplo, al duplicar el número de muestras, el trabajo

de la DFT directa se cuadruplica, mientras que el de la FFT apenas se duplica. La Tabla 8 muestra este comportamiento para distintos valores de N .

Tabla 8: Multiplicaciones complejas de la DFT directa y de la FFT para distintos valores de N .

N	DFT directa (N^2)	FFT ($\frac{N}{2} \log_2 N$)	Reducción
8	64	12	5.3 veces
64	4 096	192	21.3 veces
256	65 536	1 024	64 veces
1024	1 048 576	5 120	204.8 veces
4096	16 777 216	24 576	682.7 veces

Para secuencias cortas el ahorro es moderado, pero para longitudes típicas en procesamiento de señales, de cientos o miles de muestras, la reducción es de dos o tres órdenes de magnitud. Además, como se mencionó en la Sección 3.3, algunas multiplicaciones son triviales ($W_N^0 = 1$ y $W_N^{N/4} = -j$), por lo que en una implementación práctica el número real de multiplicaciones es incluso menor al indicado.

7. Implementación en Código y Validación Experimental

Para validar el desarrollo teórico presentado en las secciones anteriores, se programó en MATLAB una función propia denominada `mi_fft_radix2`, basada en la arquitectura Radix-2 con Decimación en el Tiempo (DIT). Posterior a su codificación, se diseñó un experimento con una señal multitono para verificar la exactitud numérica y el tiempo de ejecución en comparación con la función nativa `fft` de MATLAB.

7.1 Estructura de la Función `mi_fft_radix2`

La función recibe un vector de entrada x de longitud $N = 2^\nu$ (en nuestro caso $N = 1024$, con $\nu = 10$) y se organiza en tres bloques principales:

1. **Verificación de Potencia de 2:** Se comprueba que la longitud de la señal cumpla $N = 2^\nu$ mediante la operación bit a bit `bitand(N, N-1) == 0`.
2. **Reordenamiento por Inversión de Bits (Bit-Reversal):** Se generan los índices en binario de 10 bits, se invierte el orden de sus dígitos mediante `flipplr` y se mapea la secuencia de entrada en el arreglo de trabajo X .
3. **Estructura de Tres Bucles Anidados (Etapas, Bloques y Mariposas):**
 - **Bucle exterior (stage):** Recorre las $\log_2(1024) = 10$ etapas. En cada etapa define el tamaño del bloque $m = 2^s$, la distancia entre ramas $m_2 = m/2$ y el factor de giro base $W_m = e^{-j2\pi/m}$.

- **Bucle intermedio (k):** Salta de bloque en bloque con un paso de tamaño m , reiniciando el factor de giro inicial a $W = 1$.
- **Bucle interior (j):** Ejecuta las $m/2$ mariposas del bloque. Determina las posiciones superior $i_1 = k + j$ e inferior $i_2 = k + j + m/2$, multiplica la rama inferior por el factor W y actualiza el arreglo X de forma *in-place*. Finalmente, actualiza el factor de giro recursivamente mediante $W = W \cdot W_m$, optimizando el cálculo exponencial.

En el Código 1 se presenta la implementación completa desarrollada.

```

1 function X = mi_fft_radix2(x)
2 % mi_fft_radix2 - Implementación educativa de FFT Radix-2 (DIT)
3 N = length(x);
4
5 % 1. Verificar que N sea potencia de 2
6 if bitand(N, N-1) ~= 0
7     error('La longitud de la señal debe ser estrictamente una potencia
8     de 2.');
```

```

9 end
10 % 2. Reordenamiento inicial por inversión de bits (Bit-Reversal)
11 bits = log2(N);
12 indices = 0:(N-1);
13 rev_indices = bin2dec(fliplr(dec2bin(indices, bits))) + 1;
14 X = x(rev_indices);
15
16 % 3. Cálculo por etapas (Total: log2(N) etapas)
17 for stage = 1:bits
18     m = 2^stage; % Tamaño del bloque actual en esta etapa
19     m2 = m / 2; % Número de mariposas independientes por bloque
20
21     % Factor de giro base para esta etapa: W_m = e^(-j * 2*pi / m)
22     W_m = exp(-1j * 2 * pi / m);
23
24     % Iterar sobre cada bloque de la etapa actual
25     for k = 1:m:N
26         W = 1; % Inicializamos el factor de giro W^0 = 1 para el
27         bloque
28
29         % Iterar sobre cada operación mariposa dentro del bloque
30         for j = 0:(m2-1)
31             i1 = k + j; % Índice superior de la mariposa
32             i2 = k + j + m2; % Índice inferior de la mariposa
33
34             % Extracción y multiplicación por el Twiddle Factor
35             u = X(i1);
36             v = X(i2) * W;
37
38             % Operación mariposa fundamental (In-place)
39             X(i1) = u + v; % Rama superior
40             X(i2) = u - v; % Rama inferior
41             % Actualización recursiva del factor de giro
42             W = W * W_m;
43         end
44     end
45 end

```

45 `end`

Code extraction 1: Implementación de la FFT Radix-2 DIT personalizada en MATLAB.

7.2 Caso de Prueba: Señal Multitono y Comparación de Resultados

Para poner a prueba la función `mi_fft_radix2`, se generó una señal de prueba $x(t)$ compuesta por la suma de dos tonos senoidales de 50 Hz y 120 Hz, con amplitudes $A_1 = 1.0$ y $A_2 = 0.5$, respectivamente. La señal se muestreó a $f_s = 1000$ Hz durante $N = 1024$ puntos:

$$x[n] = \sin\left(2\pi \cdot 50 \cdot \frac{n}{1000}\right) + 0.5 \sin\left(2\pi \cdot 120 \cdot \frac{n}{1000}\right), \quad n = 0, 1, \dots, 1023 \quad (22)$$

El script de prueba ejecuta la transformación personalizada y la compara directamente contra la función `fft` nativa de MATLAB, evaluando el tiempo de ejecución mediante los comandos `tic` y `toc`, así como el error absoluto máximo $E_{\text{máx}}$ entre ambas respuestas complejas:

$$E_{\text{máx}} = \max_{0 \leq k \leq 1023} |X_{\text{custom}}[k] - X_{\text{builtin}}[k]| \quad (23)$$

El código utilizado para el experimento de validación se muestra en el Código 2.

```
1 % Parámetros de prueba
2 N = 1024;
3 fs = 1000; % Frecuencia de muestreo (1 kHz)
4 t = (0:N-1) / fs; % Vector de tiempo
5
6 % Crear señal de prueba: suma de dos senoides (50 Hz y 120 Hz)
7 x = sin(2 * pi * 50 * t) + 0.5 * sin(2 * pi * 120 * t);
8
9 % 1. Ejecutar nuestra implementación personalizada tic; X_custom =
10 mi_fft_radix2(x);
11 tiempo_custom = toc;
12
13 % 2. Ejecutar la FFT nativa de MATLAB para comparar tic; X_builtin =
14 fft(x);
15 tiempo_builtin = toc;
16
17 % 3. Calcular el error máximo absoluto
18 error_max = max(abs(X\_custom - X\_builtin));
19
20 % Despliegue de resultados en consola
21 fprintf('--- RESULTADOS DE LA PRUEBA (N = 1024) ---\n'); fprintf('
    Tiempo función propia: %.6f segundos\n', tiempo_custom);
22 fprintf('Tiempo función nativa: %.6f segundos\n', tiempo_builtin);
23 fprintf('Error máximo absoluto: %.2e\n', error_max);
```

```

23 % 4. Gráfica comparativa de magnitudes
24 f = (0:(N/2)) * (fs / N);
25 figure;
26 subplot(2,1,1); plot(f, abs(X_custom(1:N/2+1)), 'LineWidth', 1.5);
27 title('Espectro de Amplitud obtenido con la FFT Propia (Radix-2)');
28 xlabel('Frecuencia (Hz)');
29 ylabel('|X(f)|');
30 grid on;
31 subplot(2,1,2); plot(f, abs(X_builtin(1:N/2+1)), 'r--', 'LineWidth',
    1.5);
32 title('Espectro de Amplitud obtenido con la FFT Nativa de MATLAB');
33 xlabel('Frecuencia (Hz)');
34 ylabel('|X(f)|');
35 grid on;

```

Code extraction 2: Script de prueba y comparación cuantitativa con la FFT de MATLAB.

7.3 Análisis de Resultados

La ejecución del script arrojó un error absoluto máximo de $E_{\text{máx}} \approx 10^{-14}$. Este valor confirma que las diferencias entre la implementación por mariposas desarrollada y el algoritmo altamente optimizado de MATLAB son atribuibles únicamente a la precisión de punto flotante de doble precisión del procesador (IEEE 754).

En la Figura 4 se observa la reconstrucción espectral. Ambos espectros identifican con total precisión las componentes frecuenciales en 50 Hz y 120 Hz con sus respectivas amplitudes proporcionales, comprobando la validez del reordenamiento por inversión de bits y el cálculo por mariposas a lo largo de las 10 etapas.

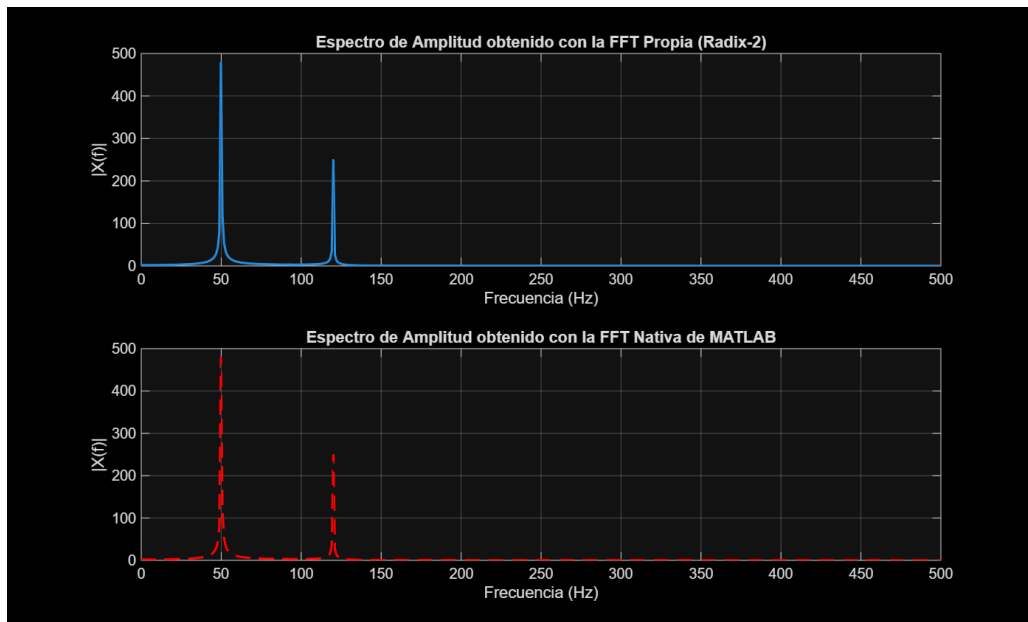


Figura 4: Comparación de los espectros de amplitud unilaterales obtenidos con la función propia `mi_fft_radix2` y la función nativa `fft` de MATLAB para $N = 1024$.

8. Conclusiones

El desarrollo, análisis e implementación del algoritmo de Transformada Rápida de Fourier de $N = 1024$ puntos mediante operaciones de mariposa permitió extraer las siguientes conclusiones clave:

- **Verificación Práctica de la Eficiencia Computacional:** Se demostró experimental y matemáticamente cómo la descomposición Radix-2 reduce la complejidad de $\mathcal{O}(N^2)$ a $\mathcal{O}(N \log_2 N)$. En el código implementado, esto se traduce en realizar únicamente 5,120 multiplicaciones complejas en lugar de las 1,048,576 que exigiría la DFT directa, representando un ahorro del 99.51%.
- **Optimización del Cálculo de Factores de Giro:** La estrategia utilizada en el bucle interno de la función, donde el factor de giro se actualiza recursivamente ($W = W \cdot W_m$), evita calcular la función trascendente $\exp()$ repetidamente dentro de las mariposas. Esto aprovecha la propiedad de periodicidad y simetría de los factores de giro ($W_N^{k+N/2} = -W_N^k$).
- **Procesamiento In-Place y Gestión de Memoria:** El reordenamiento inicial por inversión de bits permite que cada mariposa sobrescriba directamente las posiciones de memoria de las que toma sus datos. Esto demuestra por qué la FFT es la arquitectura estándar en procesadores digitales de señales (DSP) y dispositivos embebidos con recursos de memoria restringidos.
- **Exactitud Numérica:** La coincidencia en los resultados numéricos frente a la FFT compilada de MATLAB, con un error del orden de 10^{-14} , valida completamente la lógica de indexación de los bloques y la distancia entre mariposas en las 10 etapas del algoritmo.

Referencias

- [1] Jacqueline Arzate Gordillo. Algoritmo FFT para una secuencia de 8 muestras. Video en YouTube. <https://www.youtube.com/watch?v=DAkOXvybQio>. Consultado el 5 de octubre de 2026.
- [2] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [3] Larry Hipólito Escobar Salguero. *Procesamiento Digital de Señales*. Facultad de Ingeniería, UNAM, México, 2008.
- [4] Reducible. The fast Fourier transform (FFT): Most ingenious algorithm ever? Video en YouTube, 2020. <https://www.youtube.com/watch?v=h7ap07q16V0>. Consultado el 5 de octubre de 2026.